

A TECHNICAL STUDY OF AGENT ARCHITECTURE

From Agent Protocols to Accountable *Inner Bodies*

A story for AI engineers: once agents can talk and use tools, the hard question moves inward — how does an agent gather context, transform it, assess the result, ask for depth, and write back with provenance.

● OBSERVE ● GATHER AXES ● TRANSFORM ● ASSESS DEPTH ● CRYSTALLIZE

A2A MCP YAML CAPABILITIES LEDGER + GRAPH ASSESSMENT VERBS

METHOD

- OBS
- GAT
- TRA
- ASS
- CRY

— THE REASONING RHYTHM // THE FIVE-STEP LENS

AXIS // RHYTHM — EACH TOPIC IS READ THROUGH THE SAME FIVE-STEP LENS

Observe the facts, gather the axes,
transform the view into an inner
architecture, assess the result, and
crystallize a practical claim.



01 Observe

Look at the visible technical fact that starts the question.

VISIBLE TECHNICAL FACT



02 Gather axes

Collect the dimensions that must stay explicit for engineering judgment.

DIMENSIONS EXPLICIT



03 Transform

Build a logical point where observations and constraints become a design move.

DESIGN MOVE



04 Assess depth

Inspect what the transformation produced and decide whether to recurse.

RECURSE OR COMMIT



05 Crystallize

Extract the claim, invariant, or practical rule that follows.

PRACTICAL RULE

GROUND

This is a presentation rhythm, but it also mirrors the proposed agentic unit of work: context selection, transformation, assessment, and accountable write.

OBSERVE

— CYCLE 1 — OBSERVE // BOUNDARY APPEARS

AXIS // BOUNDARY — PROTOCOLS OPEN A NEW APERTURE INTO AGENT SYSTEMS

● OBS

○ GAT

○ TRA

○ ASS

○ CRY

Protocols create a new aperture into agent systems — discovery, skills, and tools become *visible at the seam*.

A2A gives agents a way to discover each other and coordinate tasks. MCP gives agents a practical way to call tools and access external systems. Together, they make the boundary visible.

A2A

Agent-to-agent

Remote agent discovery, skill declaration, messages, tasks, artifacts, and collaboration across agent boundaries.

MCP

Tools & resources

Tool, resource, and prompt exposure. The agent can reach into practical external systems.

GAP

The opening

The protocols do not specify how the remote agent internally remains accountable while acting.

CLAIM //

The boundary is now protocolized. The inner body is still an architectural choice.

GATHER

● OBS

● GAT

○ TRA

○ ASS

○ CRY

— CYCLE 1 — GATHER AXES // OUTSIDE VS INSIDE

AXIS // BOUNDARY ↔ INNER — SEPARATE EXTERNAL PROTOCOL SURFACES FROM INTERNAL ACCOUNTABILITY SURFACES

The boundary gives us *axes*, not a complete architecture.

Before designing the inner system, separate the external protocol surfaces from the internal accountability surfaces.

AXIS	BOUNDARY MANIFESTATION	INTERNAL QUESTION
Discovery	Agent Card, skills, endpoints, authentication.	What internal capabilities actually satisfy the advertised skill?
Delegation	Tasks, messages, artifacts, remote collaboration.	How does a task become a local center of transformation?
Tools	MCP tools with schemas and structured outputs.	How are tool calls selected, constrained, recorded, and assessed?
Opacity	The remote agent may remain implementation-private.	How can opacity coexist with internal audit and explainability?
Commitment	The remote side returns artifacts or status.	Who has authority to accept that artifact as a local fact?

CYCLE · 1

FIVE AXES · BOUNDARY MANIFESTATION ⊥ INTERNAL QUESTION

↓ TRANSFORM

TRANSFORM

— CYCLE 1 — TRANSFORM // TASK → CENTER

AXIS // LOCALIZATION — A PROTOCOL TASK BECOMES A LOCAL TRANSFORMATION CENTER

The external task must become a *scoped center* that pulls context, invokes capabilities, assesses the result, and posts back to the substrate.

- OBS
- GAT
- TRA
- ASS
- CRY

A2A task

EXTERNAL REQUEST, REMOTE CONTEXT,
ARTIFACT EXPECTATION

Task center

LOCAL SHELL WHERE THE WORK BECOMES
INSPECTABLE

Capabilities

YAML VESSELS, SKILLS, MCP
BINDINGS, HUMAN ROUTES

Ledger entry

RESULT, CREDITS, AUTHORITY,
RESIDUAL

CLAIM //

Protocol interoperability becomes useful when each incoming task is converted into an accountable internal event.

CRYSTAL

- OBS
- GAT
- TRA
- ASS
- CRY

— CYCLE 1 — CRYSTALLIZE // FIRST CLAIM

AXIS // CLAIM I — PROTOCOLS CONNECT AGENTS; SUBSTRATES MAKE THEM ACCOUNTABLE

First claim: protocols connect agents, but *substrates* make them accountable.

A2A and MCP solve crucial boundary problems. The next layer is the accountable body that makes each boundary event traceable, assessable, and governable.

DO NOT

Hide the body

Capabilities should be declared and governed, not buried in a single orchestration prompt.

DO NOT

Trust the artifact alone

A returned result must still be locally assessed before it becomes a committed fact.

DO NOT

Add audit later

The write path is the moment to enforce provenance, authority, and residual capture.

TRANSITION

Once the boundary event enters the body, we need a place where all transformations can meet without becoming **one mutable memory**.

OBSERVE

— CYCLE 2 — OBSERVE // MEMORY DRIFTS

AXIS // SUBSTRATE — INSIDE THE AGENT, "MEMORY" IS TOO VAGUE TO BE THE FOUNDATION

Inside the agent, "*memory*" is too vague to be the foundation.

Agents tend to accumulate vector stores, caches, scratchpads, task queues, graph tables, and audit logs. The more capable the system becomes, the more dangerous this drift gets.

DRIFT

State drifts

The graph says one thing, the trace says another, and the task queue has its own truth.

LEAK

Context leaks

Prompts silently carry data that was never made accountable as an input.

LATE

Audit arrives late

Logging after action cannot prove why the action had authority to happen.

CLAIM //

The foundation should not be memory. It should be a shared information substrate.

- OBS
- GAT
- TRA
- ASS
- CRY

GATHER

— CYCLE 2 — GATHER AXES // FIVE FACES, ONE TRUTH
AXIS // FACES OF SUBSTRATE — EACH LAYER HAS A DIFFERENT JOB

The substrate has several faces, but *one source of truth*.

We pause to separate the layers that are often collapsed into "agent memory". Each one has a different job.

AXIS	ROLE	PRACTICAL RULE
Ledger	Append-only history of ingests, outputs, decisions, reversals, and failures.	Only entries are source of truth.
Graph	Current relation structure over claims, entities, questions, tools, and results.	Graph is a projection, not a second truth.
Projection	Scoped view for a model, tool, human, or remote agent.	Execution sees a local shell, not the entire world.
Resource ledger	Compute, latency, token cost, tool cost, and human minutes.	Cost constrains action but is not semantic truth.
Authority	Who or what can write which kind of entry.	Permission is part of the architecture, not an afterthought.

— CYCLE 2 — TRANSFORM // ONE LEDGER, MANY WORLDS

AXIS // PROJECTION — ONE LEDGER CAN GENERATE MANY WORKING WORLDS

One ledger can generate *many working worlds*.

The agent does not move raw reality into every step. It generates local projections from a durable substrate, then writes the transformed result back as a new entry.

Append-only ledger

EVERYTHING THAT HAPPENED, UNDER
WHOSE AUTHORITY

Graph projection

ENTITIES, RELATIONS, CONFLICTS,
RESIDUAL QUESTIONS

Execution shell

SELECTED CONTEXT FOR ONE
TRANSFORMATION CENTER

Decision view

FULL DISCLOSURE AT COMMIT POINTS

```
projection:
  center: task.current
  include:
    - SOURCED claims relevant to selector profile
    - DERIVED graph links within depth 1
    - GENERATED liabilities not yet resolved
    - resource budget and authority constraints
  exclude:
    - unrelated private state
    - uncredited outputs
```

CRYSTAL

- OBS
- GAT
- TRA
- ASS
- CRY

— CYCLE 2 — CRYSTALLIZE // SECOND CLAIM

AXIS // CLAIM II — TRACES ARE THE DATA, NOT SYNCHRONIZED WITH IT

Second claim: do not synchronize traces with data. *Make traces the data.*

If the ledger is primary, the graph, context, task state, and review surface become rebuildable views. This turns accountability from a feature into a structural property.

INVARIANT

No silent results

No result exists without a ledger entry.

INVARIANT

No orphan edges

No graph edge exists without an entry that explains how it appeared.

INVARIANT

No hidden liabilities

No decision view hides generated liabilities at the point of commitment.

GROUND

Event sourcing and CQRS provide the systems precedent: **immutable history first, read models second.**

SPECIMEN // AGENT·ARCH·001

11 / 22 — C3 · OBSERVE

OBSERVE

● OBS

○ GAT

○ TRA

○ ASS

○ CRY

CYCLE 3 — OBSERVE // THE UNIT QUESTION

AXIS // UNIT OF WORK — A TOOL CALL IS TOO SMALL TO BE THE ARCHITECTURAL UNIT

A tool call is too small to be the unit of *agent architecture*.

The visible action may be a model call, function call, MCP tool, human review, or remote A2A task. But the meaningful architectural unit is larger.

SELECT

Selection

What was allowed into the local shell?

EXECUTE

Execution

Who or what performed the transformation?

JUDGE

Judgment

Why did the system accept, retry, ask, escalate, or abort?

CLAIM //

The unit is not "call the tool". The unit is "transform a scoped projection and decide what may happen next".

CYCLE 3

SELECTION · EXECUTION · JUDGMENT

↓ GATHER AXES

- GATHER
- OBS
 - GAT
 - TRA
 - ASS
 - CRY

— CYCLE 3 — GATHER AXES // VESSEL CONTRACT

AXIS // VESSEL — EACH CAPABILITY IS A VESSEL WITH A COMMON CONTRACT

Each capability is a *vessel* with a common contract.

A vessel is an isolated function that can enter the shared substrate, pull what it needs, work inside a shell, judge the result, and return an accountable entry.

AXIS	QUESTION	DESIGN ARTIFACT
Resolve	How does the vessel pull inputs?	Selectors, cardinality, attribute ontology.
Act	What executor performs the transformation?	Code, LLM, MCP tool, human task, A2A agent.
Assess	What did the vessel learn about the result?	Verb table, residual, confidence, novelty, authority.
Post	How does the result return?	Balanced ledger entry, graph delta, write-off, question.

- STATIC YAML
- DYNAMIC SKILLS
- MCP TOOLS
- HUMAN ROUTES
- REMOTE AGENTS

— CYCLE 3 — TRANSFORM // THE VESSEL LOOP

AXIS // VESSEL LOOP — EXECUTION AND ACCOUNTABILITY BECOME ONE SHAPE

The vessel loop makes execution and accountability *one shape*.

Different executors can be swapped without changing the surrounding architecture, because the vessel contract stays stable.



```
capability:
  id: coverage_probe
  resolve:
    selector: sourced_policy_claims
    cardinality: at-least-one
  act:
    executor: mcp_tool_or_model
  assess:
    verbs: [PROCEED, RETRY, ASK, ESCALATE, ABORT]
  post:
    requires_balanced_entry: true
```

WHY THIS MATTERS

The vessel can be static in YAML or dynamically attached as a skill. Either way, it must use the same entry, assessment, and authority rules.

ASSESS

● OBS

● GAT

● TRA

● ASS

○ CRY

— CYCLE 3 — ASSESS DEPTH // VERB TABLE

AXIS // VERB TABLE — ASSESSMENT IS WHERE THE SYSTEM DECIDES TO GO DEEPER

Assessment is where the system decides whether to *go deeper*.

The result is inspected from inside the current logical point: what was intended, what was selected, what was produced, what remains unresolved, and what has authority to move forward.

VERB	WHEN IT APPEARS	RULE
PROCEED	The result is sufficient for the current commitment height.	Log as an explicit decision.
RETRY	The same vessel may improve the result.	Require measurable residual reduction.
ASK	The vessel finds a deficit that requires another projection.	Write the question as a first-class entry.
ESCALATE	Novelty, irreversibility, or missing authority exceeds policy.	Route to human or higher authority.
ABORT	The state is unknown, unsafe, or outside contract.	Fail closed.

CYCLE · 3

PROCEED · RETRY · ASK · ESCALATE · ABORT

↓ CRYSTALLIZE

CRYSTAL

- OBS
- GAT
- TRA
- ASS
- CRY

— CYCLE 3 — CRYSTALLIZE // THIRD CLAIM

AXIS // CLAIM III — CAPABILITIES ARE VESSELS, NOT FREE-FLOATING TOOLS

Third claim: capabilities are *vessels*, not free-floating tools.

A vessel has hands, but it also has a contract. It can use MCP, call a model, ask a human, or delegate through A2A. What matters is how it enters and leaves the shared substrate.

INPUT

Input discipline

Selectors define what the vessel may see.

EXEC

Execution discipline

Executors are interchangeable only behind explicit schemas.

OUTPUT

Output discipline

Every output returns through a balanced write.

TRANSITION

Once assessment can produce questions, the architecture stops being a fixed workflow and becomes a **bounded route-growing system**.

OBSERVE

● OBS

○ GAT

○ TRA

○ ASS

○ CRY

— CYCLE 4 — OBSERVE // DEFICITS AS OPENINGS

AXIS // DEFICIT — DEFICITS CREATE ROUTES THE ORIGINAL WORKFLOW DID NOT DRAW

Deficits create routes the original workflow *did not draw*.

A capable agent will encounter missing sources, conflicting evidence, insufficient authority, novelty, and unclear user intent. The question is not whether this happens. The question is how the system handles it.

STATIC

Static workflows stop

If the path was not drawn, the system either fails or improvises invisibly.

UNBOUND

Unbounded agents drift

If every deficit becomes unrestricted exploration, the system loses control.

BOUNDED

Bounded agents ask

The deficit becomes a logged question routed through explicit controls.

CLAIM //

A question is not merely clarification. It is a controlled way to open a new route.

— CYCLE 4 — GATHER AXES // ROUTES & LIMITS

AXIS // ROUTING - QUESTIONS NEED ROUTING RULES AND GROWTH LIMITS

Questions need *routing rules* and growth limits.

— CYCLE 4 — TRANSFORM // THE DEFICIT TUNNEL

AXIS // EMERGENT ROUTE — A DEFICIT BECOMES A TUNNEL THROUGH THE SUBSTRATE

A deficit becomes a *tunnel* through the substrate.

The system can grow a path through internal vessels, external agents, tools, and human judgment. The route is emergent, but the movement is bounded.

Logged question

CREATED BY ASSESS WHEN THE CURRENT SHELL CANNOT HONESTLY COMMIT THE RESULT.

M1

Re-enter

Same vessel, changed selector profile, measurable progress.

M2

Branch

Several vessels inspect the deficit from different projections.

M3

Delegate

A2A remote agent or human authority receives the deficit.

```
question_entry:
  created_by: assess.coverage_probe
  deficit: "source evidence insufficient for commit"
  route_candidates:
    - retry:same_vessel.selector_broadened
    - peer:source_lookup_vessel
    - external:a2a_research_agent
  stop_condition: residual_decreases_or_escalate
```

- OBS
- GAT
- TRA
- ASS
- CRY

— CYCLE 4 — ASSESS & CRYSTALLIZE // BOUNDED EMERGENCE
AXIS // CLAIM IV — EMERGENCE IS USEFUL ONLY WHEN IT IS BOUNDED

Fourth claim: emergence is useful only when it is *bounded*.

The system should be able to discover a route through a problem, but every route must remain inspectable, reversible where possible, and constrained by authority.

MAY GROW

What may grow

- New questions.
- New routes through capabilities.
- New external delegations.
- New graph links and projections.
- New dynamically attached skills.

MUST HOLD

What must not drift

- Selector contracts.
- Verb table and fail-closed defaults.
- Authority model.
- Resource budget.
- Balanced write rules.

GROUND

This is a **controlled dynamic graph**: back-edges are allowed only when they reduce a declared residual or escalate.

SYNTHESIS

— FINAL CYCLE — CRYSTALLIZE // THE PRACTICAL MODEL

AXIS // MODEL — THE ACCOUNTABLE AGENT IN ONE STACKED VIEW

The practical model in *one view*.

An accountable agent is a protocol-connected body of vessels working over a shared substrate, where each transformation can ask, retry, escalate, or commit with provenance.

Boundary	A2A for agent-to-agent tasks. MCP for practical tools and resources.
Body	Declarative vessels: YAML capabilities, skills, schemas, policies, executor bindings.
Substrate	Append-only ledger, graph projection, working views, resource ledger, decision surfaces.
Motion	Resolve context, act through an executor, assess depth, post accountable entries.
Growth	Questions open new routes while selector, authority, budget, and write rules bound the system.

CLAIMS

- OBS
- GAT
- TRA
- ASS
- CRY

— CLAIMS TO TAKE AWAY // FOR AI ENGINEERING TEAMS

AXIS // FIVE CLAIMS – CONCLUSIONS THAT CRYSTALLIZE FROM THE CYCLES

Five practical claims for *AI engineering teams.*

CLAIM I

Protocols are boundaries

A2A and MCP are necessary, but they do not define the internal body of the agent.

CLAIM II

Trace should be substrate

Do not synchronize logs with data. Build projections from an append-only entry history.

CLAIM III

Capabilities are vessels

Every capability should resolve, act, assess, and post through one contract.

CLAIM IV

Questions are architecture

A question is a first-class way to route deficits, not a hidden model thought.

CLAIM V

Generated output is liability

A model result is useful, but it is not a committed fact until sourced, validated, or decided.

RULE

Operating rule

Let the system discover a path while every move remains bounded, inspectable, and accountable.

GROUND

- OBS
- GAT
- TRA
- ASS
- CRY

— GROUNDING NOTES // TECHNICAL ANCHORS BEHIND THE LENS
AXIS // ANCHORS — ALGEBRA · TOPOLOGY · STRUCTURAL INFORMATION SYSTEMS

Technical *anchors* behind the lens.

I · ALGEBRA

Algebra

Typed operators. A vessel behaves like an operator from selected inputs to posted outputs.

Composition. Operators compose only when schemas, authority, and side effects are explicit.

Differential. Assess compares intended, selected, produced, authorized, and residual state.

Liability. Generated content is a pending obligation, not a conserved fact.

II · TOPOLOGY

Topology

Local charts. Each execution shell is a local projection over the shared substrate.

Neighborhoods. Selectors define which part of the graph is visible from the current center.

Edges. Questions create new routes in the work graph.

Back-edges. Retry is allowed only with a progress measure or escalation.

III · SYSTEMS

Structural information systems

Event sourcing. Immutable entries first, read models second.

CQRS. Write model and read projections serve different purposes.

Blackboard systems. Capabilities coordinate through shared structured memory.

Data provenance. Derivation edges make lineage inspectable.

REFERENCES

Github: [vestacore / semantic-field](#) · Protocols: [A2A docs](#), [A2A & MCP](#), [A2A discovery](#), [MCP spec](#), [MCP tools](#).